



CEOI 2006 竞赛题交流

清华大学 王宏 陈家炜

CEOI2006-Day2 题目1 连接 (Connect)

输入: **stdin**

输出: **stdout**

时间限制: **0.5 sec**

内存限制:(堆)**32 MB**, (栈)**8 MB**



Connect-问题描述

- 给定一个 $R \times C$ 大小的迷宫，其中 R, C 均为奇数
- 迷宫中坐标为两个奇数的点不能通过，称为障碍，迷宫中其他不能通过的点统称为墙壁
- 坐标为两个偶数的点可以通过，称为房间，迷宫中其他可通过的点称为走廊。迷宫有边界
- 迷宫中放置有若干个物品(偶数个)，物品一定放置在房间中
- 问如何将这些物品配对可以使得所有物品对之间路径的总和最小，任两条路径不能相交，并求路径



Connect输入与数据范围

- 第一行输入为两个整数：
R与C ($5 \leq R \leq 25$, $5 \leq C \leq 80$)
其中R为行数，C为列数，R，C均为奇数。
- 接下来的R行每行包括C个字符，每个字符都是'+'、'|'、'-'、空格或'X'中的一个，分别表示障碍及两种墙壁、空格表示房间或可通过的走廊，X表示房间中的物品。



Connect-输出

- 输出的第一行是一个整数，即最小分值。
- 接下来的R行描述了游戏进行之后棋盘的状态。棋盘的格式应该和输入时一样，而每个路径上的点都应该用' .'来表示。
- 注意：如果问题有多解，那么仅需输出一个。



Connect需要注意的细节

- 输入时带有空格，需要留意
- 一定要先读入第一行R与C后面的换行符



Connect-动态规划

- 本题输入规模不大，可以通过遍历整个图的状态来搜索到最优解
- 注意搜索策略和利用动态规划减少搜索中的重复



Connect--算法设计

- 下图1所示的为迷宫中的一个“房间”以及它与它相邻的四个走廊可能的11种状态。

1.	2.	3.	4.	5.	
+ +	+ . +	+ +	+ +	+ +	
	X	X.	X	.X	
+ +	+ +	+ +	+ . +	+ +	
6.	7.	8.	9.	10.	11.
+ +	+ . +	+ . +	+ +	+ +	+ . +
. . .	.	. .	. .	. .	. .
+ +	+ . +	+ +	+ . +	+ . +	+ +



Connect--算法设计

- 对于一个含有物品的房间及其邻近四点，其状态必然是2~5中的一种，否则不满足题意
- 对于一个空的房间及其邻近四点，其状态必然是1以及6~11中的一种

1.	2.	3.	4.	5.	
+	+	+	+	+	
+	+	+	+	+	
	X	X.	X	.X	
+	+	+	+	+	
6.	7.	8.	9.	10.	11.
+	+	+	+	+	+
+	+	+	+	+	+
.	.	.	.	.	.
+	+	+	+	+	+
+	+	+	+	+	+



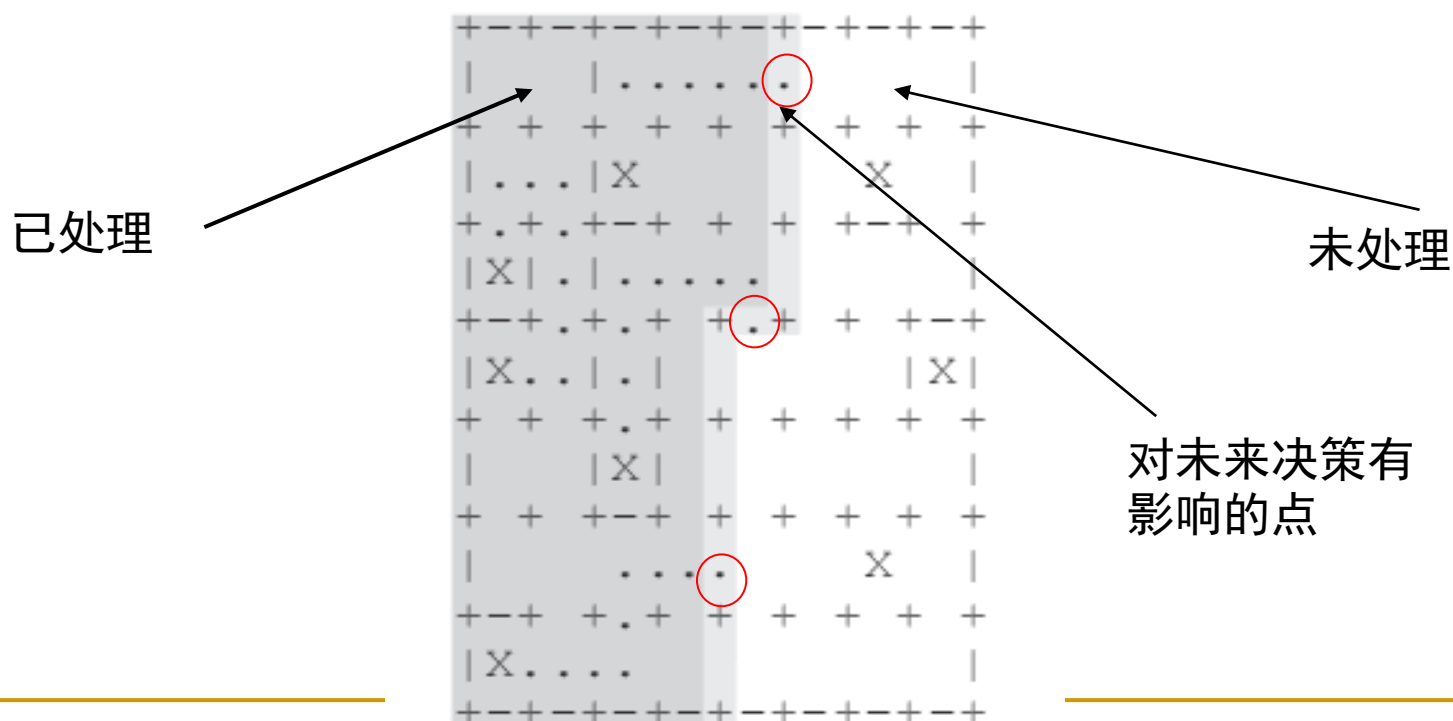
Connect--算法设计

- 由于路径必然是连续的，如果一个房间四周的走廊中含有路径，那么这个房间必然要处于6~11描述的6种状态之一
- 我们可以从迷宫的左上角开始，逐列枚举迷宫房间状态，最终求得最优解



Connect--算法设计

- 每次添加路径后，影响以后决策的只有与尚未处理的房间相邻的走廊的状态，如下图





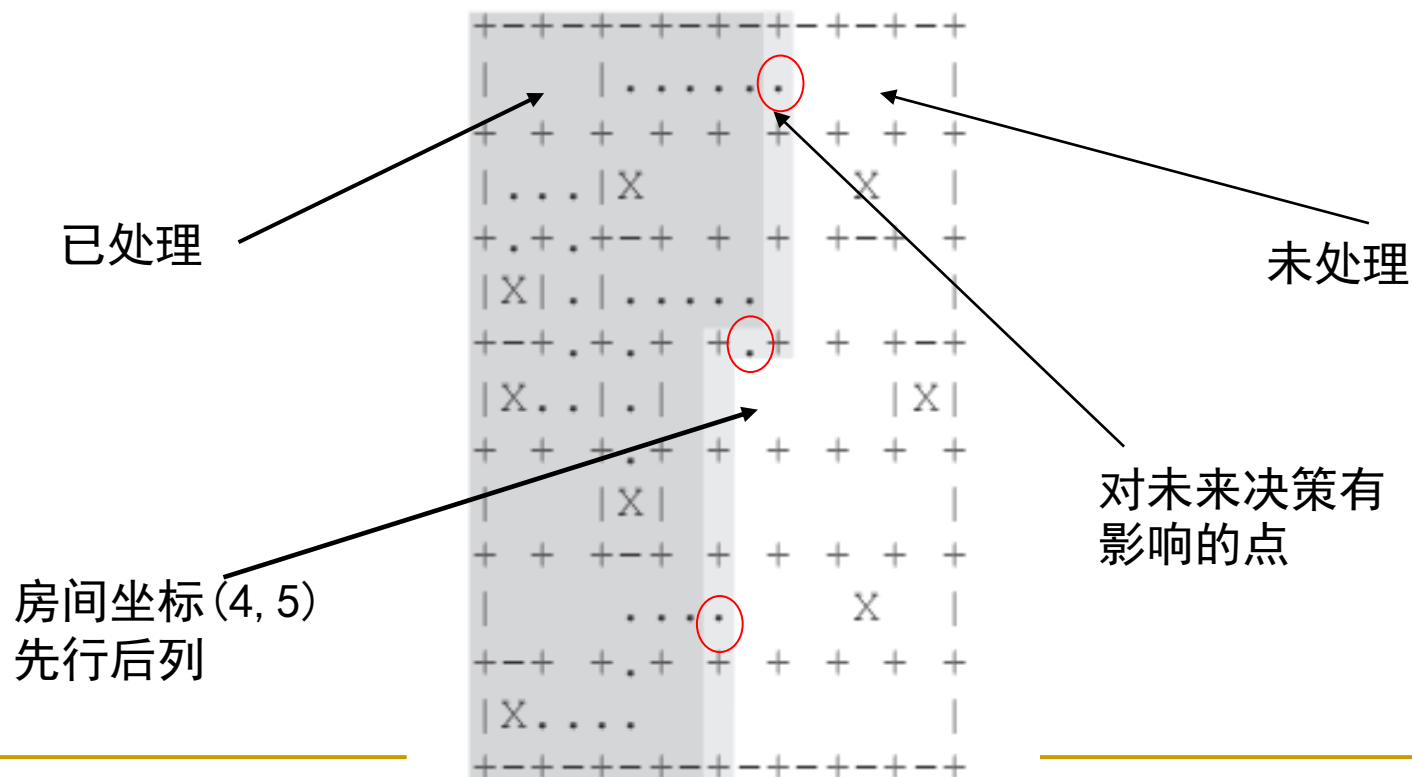
Connect--算法设计

- 迷宫至多25行，这些点的数目不会超过 $R/2+1=13$ ，每个点状态仅有两种，可以用一个BitArray，在这里是整形变量来描述。因此可以用三个整数来描述当前搜索状态
- 即：当前搜索房间的坐标(因为仅处理房间，所以坐标是房间编号)、当前图形与未处理房间相邻走廊的状态



Connect--算法设计

■ 仍以下图为例，状态为 $(4,5,(10010010)_2)$





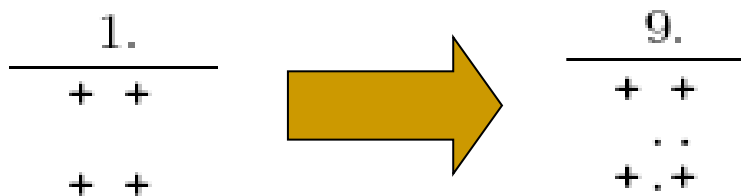
Connect--算法设计

- 搜索时记录每一种状态的路径最小值，如果在后面的搜索中遇到比这个值大的情况，则立即舍去
- 搜索时要注意不能重复搜索物品，可以考虑在处理物品或者处理物品临近的走廊后将物品置于已处理的状态



Connect--算法设计

- 还要注意四周走廊都没有路径的空房间，事实上也有可能在将来的某个时刻成为路径的一部分，即下图所示过程：



- 所以搜索的时候不要遗漏



Connect--算法分析

- 房间总量大约是 $N*M(N=(R/2), M=(C/2))$
- 总状态数是 $N*M*2^{(N+1)}$
- 这些状态需要遍历，所以算法复杂度为 $O(N*M*2^{(N+1)})$
- 由于对每一个状态还需要储存它的最优解
- 所需要的空间为 $O(N*M*2^{(N+1)})$



Connect--算法实现

```
■ int main()
■ {
■     //输入数据
■     Solve(1,1,0,0); //调用递归，进行动态规划，注意初始条件
■     //作必要处理后输出数据。
■     printf("%d\n",MinScore+square_num/2);
■     for(i=0;i<R;++i){
■         for(j=0;j<C;++j){
■             if (MinBoard[i][j]=='Y') MinBoard[i][j]='X';
■             printf("%c",MinBoard[i][j]);
■         }
■         printf("\n");
■     }
■     return 0;
■ }
```




Connect--算法实现

- **void Solve(int r,int c,int Score,long Status){**
- //如果已抵达右侧边界，则结束递归，判断当前值是否优于已的结果，确认是否保留
- //计算出下一步的行、列位置
-
- **if (Board[r][c]=='X') {**
- //先处理房间内有物品的四种情况，并计算下一个状态，
- 递归
- **}**
- **else if(Board[r][c]==' '){**
- //再处理房间内无物品的七种情况，并计算下一个状态，
- 递归
- **}**
- **else if (Board[r][c]=='Y') {**
- //前面的处理中注意将已有路径相连的物品设置为已处理，
- 这
- //里用 “Y”来表示，并计算下一个状态，递归
- **}**
- **}**
- **}**